Laboratory 5
Textures

Textures

Textures come in two broad types: procedural and image. Procedural textures are generated on the fly based on some algorithm. There are "equations" for wood, marble, asphalt, stone, and so on. Nearly any kind of material can be reduced to an algorithm and hence drawn onto an object, as shown in Figure 1.

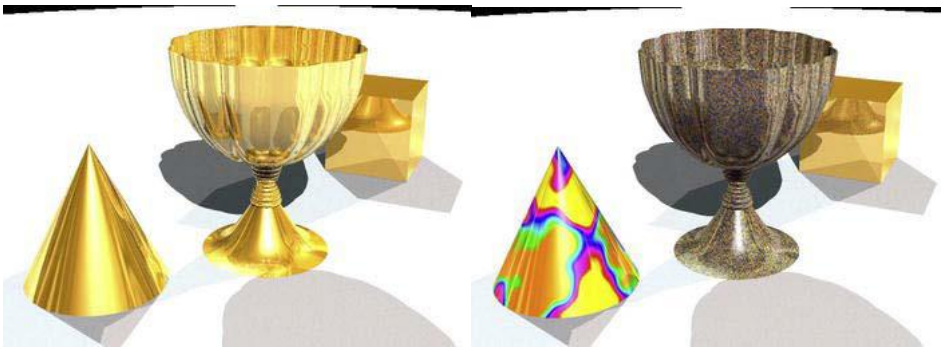

Figure 1: The chalice on the left is polished gold, while on the right uses a procedural texture to look like gold ore, while the cone looks like marble

Procedural textures are very powerful because they can produce an infinite variety of scalable patterns that can be enlarged to reveal increasingly more detail, as shown in Figure . Otherwise, this would require a massive static image.

Figure 2: Close-up on the goblet at the right in Figure 1. Notice the fine detailing that would need a very large image to accomplish

Procedural textures, and to a lesser degree, image textures, can be classified in a spectrum of complexity from random to structured. Random, or *stochastic* textures, can be thought of as

"looking like noise", like a fine-grained material such as sand, dust, gravel, the grain in paper, and so on. Near stochastic could be flames, grass, or the surface of a lake. On the other hand, *structured* textures have broad recognizable features and patterns. A brick wall, wicker basket, or plaid would be structured.

Image Textures

As referenced earlier, image textures are just that. They can serve duty as a surface or material texture such as mahogany wood, steel plating, or leaves scattered across the ground. If done right, these can be seamlessly tiled to cover a much larger surface than the original image would suggest. And because they come from real life, they don't need the sophisticated software used for the procedural variety. Figure 5–4 shows the chalice scene, but this time with wood textures, mahogany for the chalice, and alder for the cone, while the cube remains gold.

Figure 3: Using real-world image textures

Besides using image textures as materials, they can be used as pictures themselves in your 3D world. A rendered image of a tablet can have a texture dropped into where the screen is. A 3D city could use real photographs for windows on the buildings, for billboards, or for family photos in a living room.

OpenGL ES and Textures

When OpenGL ES renders an object, it draws each triangle and then lights and colorizes it based on the three vertices that make up each face. Afterward, it merrily goes about to the next one. A texture is nothing more than an image. As you learned earlier, it can be generated on the fly to handle context-sensitive details (such as cloud patterns), or it can be a `.jpg`, `.png`, or anything else. It is made up of pixels, of course, but when operating as a texture, they are called *texels*. You can think of an OpenGL ES texture as a bunch of little colored "faces" (the texels), each of the same size and stitched together in one sheet of, say, 256 such "faces" on a side. Each face is the same size as each other one and can be stretched or squeezed so as to work on surfaces of any size or shape. They don't have corner geometry to waste memory storing xyz values and can come in a multitude of colors. And of course they are extraordinarily versatile.

Like your geometry, textures have their own coordinate space. Where geometry denotes locations of its many pieces using the Cartesian coordinates x, y, and z, textures use s and t. The process that applies a texture to some geometric object is called UV mapping. (s and t are used only for OpenGL world, whereas others use u and v)

So, how is this applied? Say you have a square tablecloth that you must make fit a rectangular table. You need to attach it firmly along one side and then tug and stretch it along the other until it just barely covers the table. You can attach just the four corners, but if you really want it to "fit", you can attach other parts along the edge or even in the middle. That's a little how a texture is fitted to a surface.

Texture coordinate space is normalized; that is, both s and t range from 0 to 1. They are unitless entities, abstracted so as not to rely on either the dimensions of the source or the destination. So, the face to be textured will carry around with its vertices s and t values that lay between 0.0 to 1.0, as shown in Figure 4.

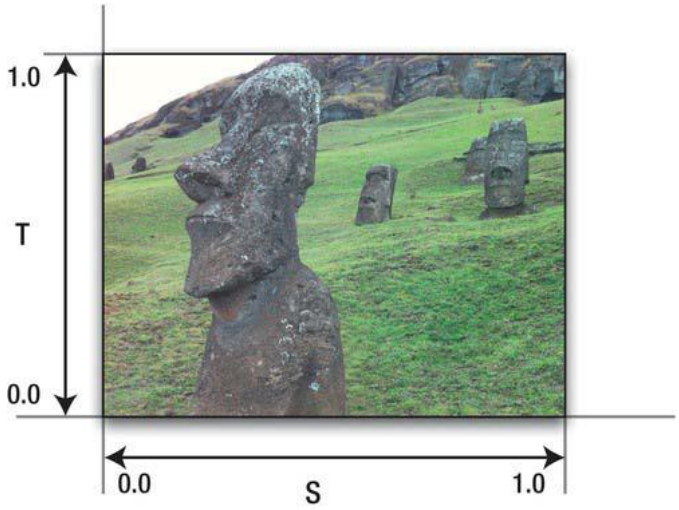

Figure 4: Using real-world image textures

In the most elementary example, we can apply a rectangular texture to a rectangular face and be done with it, as illustrated in Figure 4. But what if you wanted only part of the texture? You could supply a `.png` that had only the bit you wanted, which is not very convenient if you wanted to have many variants of the image. However, there's another way. Merely change the s and t coordinates of the destination face. Let's say all you wanted was the upper-left quarter of the Easter Island statue. All you need to do is to change the coordinates of the destination, and those coordinates are based on the proportion of the image section you want, as shown in Figure 5. That is, because you want the image to be cropped halfway down the S-axis, the s coordinate will no longer go from 0 to 1 but instead from 0 to 0.5. And the t coordinate would then would go from 0.5 to 1.0. If you wanted the lower-left corner, you would use the same 0 to 0.5 ranges as the s coordinate.

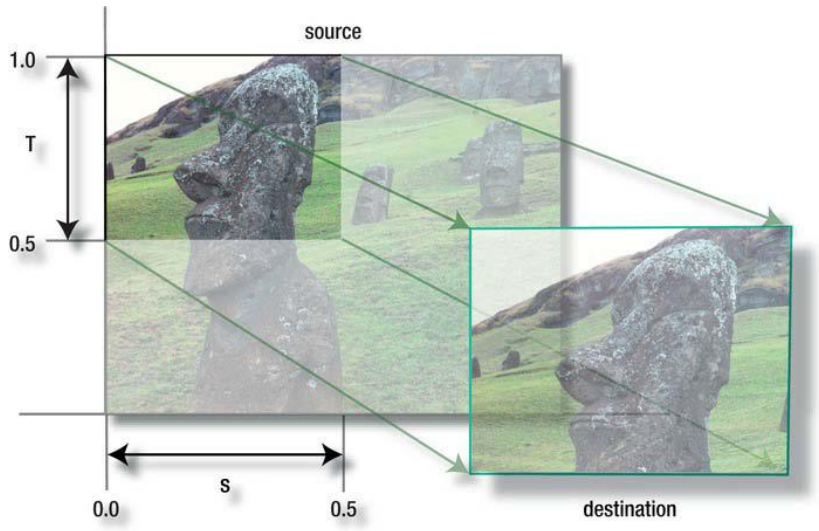

Figure 5: Clipping out a portion of the texture by changing the texture coordinates

Also note that the texture coordinate system is resolution independent. That is, the center of an image that is 512 on a side would be $(0.5, 0.5)$, just as it would be for an image 128 on a side.

Textures are not limited to rectilinear objects. With careful selections of the *st* coordinates on your destination face, you can do some of the more colorful shapes depicted in Figure 6.

Figure 6: Mapping an image to unusual shapes

If you keep the image coordinates the same across the vertices of your destination, the image's corners will follow those of the destination, as shown in Figure 7.

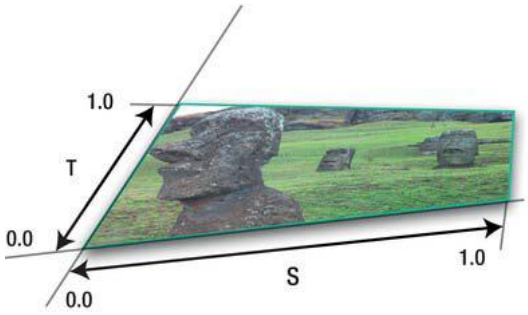

Figure 7: Distorting images can give a 3D effect on 2D surfaces

Textures can also be *tiled* so as to replicate patterns that depict wallpaper, brick walls, sandy beaches, and so on, as shown in Figure 8. Notice how the coordinates actually go beyond

the upper limit of 1.0. All that does is to start the texture repeating so that, for example, an s of 0.6 equals an s of 1.6, 2.6, and so on.

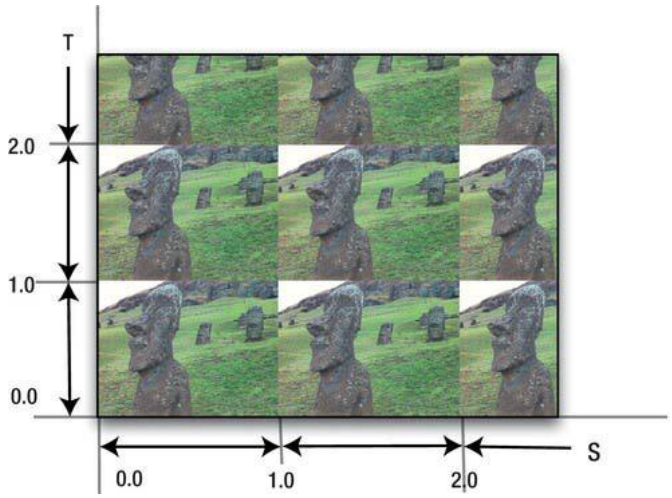

Figure 8: Tiled images are useful for repeated patterns such as those used for wallpaper or brick walls

Besides the tiling model shown in Figure 8, texture tiles can also be "mirrored", or clamped. Both are mechanisms for dealing with s and t outside of the 0 to 1.0 range.

Mirrored tiling is like repeat but merely flips columns/rows of alternating images, as shown in Figure 9 left. Clamping an image means that the last row or column of texels repeats, as shown in Figure 9 right. Clamping looks like a total mess with the sample image but is useful when the image has a neutral border. In that case, you can prevent any image repetition on either or both axes if s or t exceeds its normal bounds.

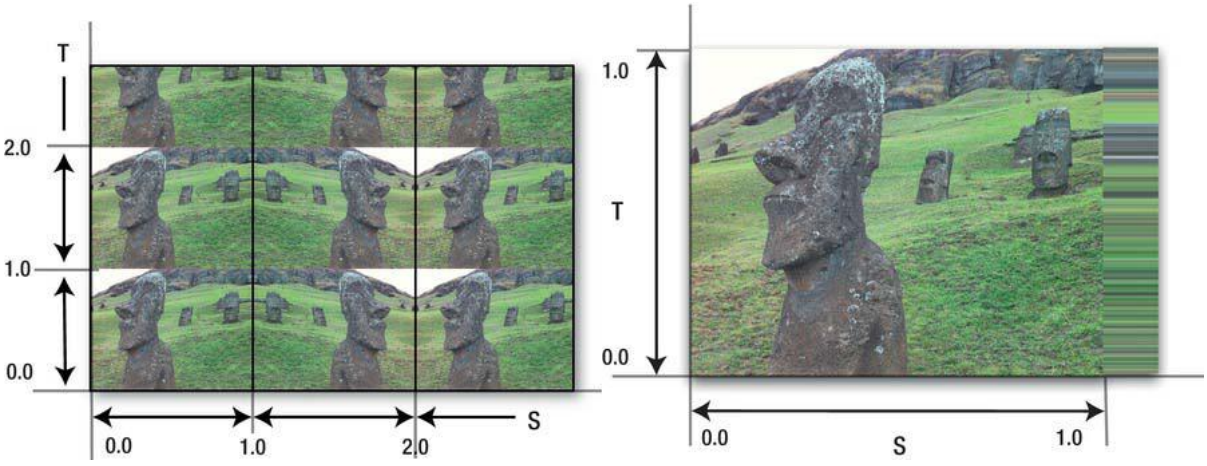

Figure 9: The left shows a mirrored-repeat for just the S axis, while the right texture is clamped

OpenGL ES, as you know by now, doesn't do *quadrilaterals*—that is, faces with four sides (as opposed to its desktop counterpart). So, we have to fabricate them using two triangles, giving us structures such as the triangle strips and fans that we experimented with in Laboratory 3. Applying textures to this "fake" quadrilateral is a simple affair. One triangle has texture coordinates of $(0, 0)$, $(1, 0)$, and $(0, 1)$, while the other has coordinates of $(1, 0)$, $(1, 1)$, and $(0, 1)$. It should make sense if you study Figure 10.

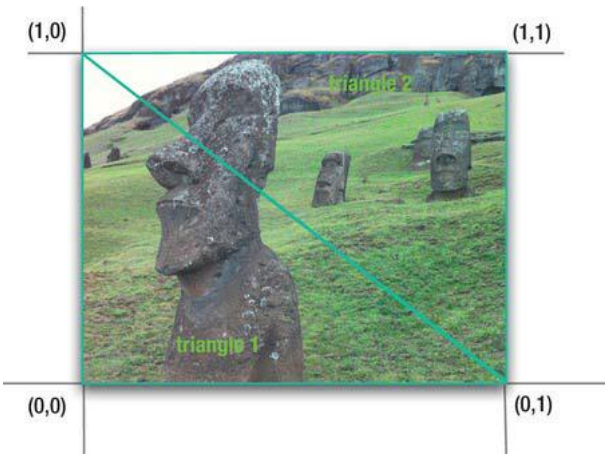

Figure 10: Placing a texture across two faces

And finally, let's take a look at how a single texture can be stretched across a whole bunch of faces, as shown in Figure 11.

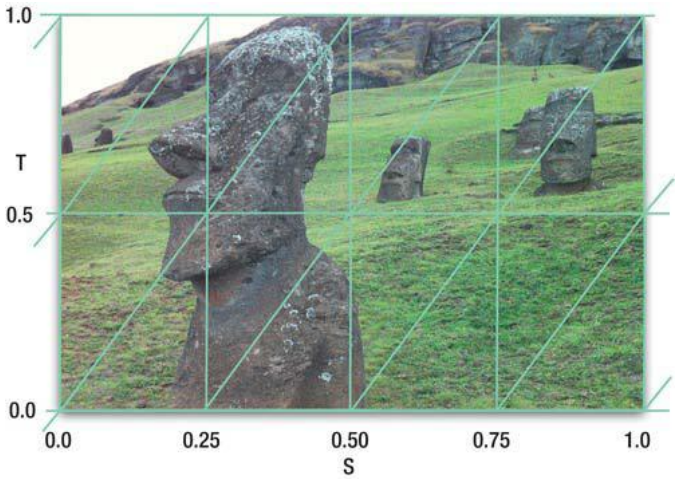

Figure 11: Stretching a texture across many faces

Generally, OpenGL can use only texture images that are power-of-two on a side.

Adding Texture to `BouncySquare`

We will apply a texture to the `BouncySquare` and then manipulate it to show some of the tricks detailed earlier, such as repeating, animating, and distortion.

We begin by creating the actual texture. This will read in an image file of any type that Android supports and convert it to a format that OpenGL can use.

Before we start, one additional step is required. Android needs to be notified about the image file that will be used as the texture. This can be accomplished by adding the image file `hedly.png` to `/res/drawable` folder. If the drawable folder doesn't exist, you may create it now and add it to the project. We will be storing texture information in integer array. Add the following line to the `Square` class:

```
private int[] textures = new int[1];
```

Add the following imports:

```
import android.graphics.*;
import android.opengl.*;
```

Now, add a method `createTexture(GL10 gl, Context contextRegf, int resource)` to the `Square` class. First, we load the Android bitmap, letting the loader handle any image format that Android can read in.

```
Bitmap image = BitmapFactory.decodeResource(contextRegf.getResources(), resource);
```

Then, `glGenTextures` gets an unused texture "name", which in reality is just a number. This ensures that each texture you use has a unique identifier. If you want to reuse identifiers, then you would call `glDeleteTextures`.

```
gl.glGenTextures(1, textures, 0);
```

After that, the texture is bound to the current 2D texture, exchanging the new texture for the previous one. In OpenGL ES, there is only one of these texture targets, so it must always be `GL_TEXTURE_2D`, whereas bigger OpenGL versions has several. Binding also makes this texture active, because only one texture is active at a time. This also directs OpenGL where to put any of the new image data.

```
gl.glBindTexture(GL10.GL_TEXTURE_2D, textures[0]);
```

Next, `GLUtils`, an Android utility class that binds OpenGL ES, and the Android APIs is used. This utility specifies the 2D texture image for the bitmap that we created above. The image (texture) is created internally in its native format based on the bitmap created.

```
GLUtils.texImage2D(GL10.GL_TEXTURE_2D, 0, image, 0);
```

Finally, we set some parameters that are required on the Android. Without them, the texture has a default "filter" value that is unnecessary at this time. The min and max filters tell the system how to handle a texture under certain circumstances where it has to be either shrunk down or enlarged to fit a given polygon.

```
gl.glTexParameterf(GL10.GL_TEXTURE_2D, GL10.GL_TEXTURE_MIN_FILTER, GL10.GL_LINEAR);
gl.glTexParameterf(GL10.GL_TEXTURE_2D, GL10.GL_TEXTURE_MAG_FILTER, GL10.GL_LINEAR);
```

The last line of the method tell Android to explicitly recycle the bitmap because bitmaps can take up a lot of memory.

```
image.recycle();
```

Before we call the `createTexture()` method, we need to get the context and the resource ID of the image (`hedly.png`). To get the context, modify the `onCreate()` method in `BouncySquareActivity` from this:

```
view.setRenderer(new SquareRenderer());
```

to the following:

```
view.setRenderer(new SquareRenderer(this.getApplicationContext()));
```

This will also require changing the constructor definition in `SquareRenderer` to the following:

```
public SquareRenderer(Context context)
{
    this.context = context;
    this.mSquare = new Square();
}
```

You will need to add the following import:

```
import android.content.Context;
```

And add an instance variable to support the new context. The context is used a little later when the image is loaded and converted to an OpenGL-compatible texture.

Now, to get the resource ID, add following to in the `onSurfaceCreated()` method in `SquareRenderer`:

```
int resid = R.drawable.hedly;
mSquare.createTexture(gl, this.context, resid);
```

The first line gets the resource of the image (`hedly.png`) that we added in the drawable folder. In the second line, we use the object of `Square` class and call the `createTexture()` method with the correct context and the resource ID of the image.

Then add the following to the instance variables in `Square`:

```
public FloatBuffer mTextureBuffer;
float[] textureCoords =
{
    0.0f, 0.0f,
    1.0f, 0.0f,
    0.0f, 1.0f,
    1.0f, 1.0f
};
```

This defines the texture coordinates. Now create the `textureBuffer` similar to `mFVertexBuffer` we created in Laboratory 2 in the square's constructor.

```
ByteBuffer tbb = ByteBuffer.allocateDirect(textureCoords.length * 4);
tbb.order(ByteOrder.nativeOrder());
mTextureBuffer = tbb.asFloatBuffer();
mTextureBuffer.put(textureCoords);
mTextureBuffer.position(0);
```

Finally, the `draw()` routine needs to be modified. After the usual

```
gl.glVertexPointer(2, GL10.GL_FLOAT, 0, mFVertexBuffer);
gl.glEnableClientState(GL10.GL_VERTEX_ARRAY);
gl.glColorPointer(4, GL10.GL_UNSIGNED_BYTE, 0, mColorBuffer);
gl.glEnableClientState(GL10.GL_COLOR_ARRAY);
```

enable the `GL_TEXTURE_2D` target. Desktop OpenGL supports 1D and 3D textures but not ES.

```
gl.glEnable(GL10.GL_TEXTURE_2D);
```

Next, blending can be enabled. Blending is where color and the destination color are blended (mixed) according to some equation that is switched on next. The blend function determines how the source and destination pixels/fragments are mixed together. The most common form is where the source overwrites the destination, but others can create some interesting effects. Since this is such a large topic, it will be covered a little later.

```
gl.glEnable(GL10.GL_BLEND);
gl.glBlendFunc(GL10.GL_ONE, GL10.GL_SRC_COLOR);
```

We must ensure that the texture we want is the current one and that the texture coordinates are handed off to the hardware.

```
gl.glBindTexture(GL10.GL_TEXTURE_2D, textures[0]);
gl.glTexCoordPointer(2, GL10.GL_FLOAT,0, mTextureBuffer);
```

And just as you had to tell the client to handle the colors and vertices, you need to do the same for the texture coordinates.

```
gl.glEnableClientState(GL10.GL_TEXTURE_COORD_ARRAY);
```

This time besides drawing the colors and the geometry using `glDrawElements`, it now takes the information from the current texture (the `texture_2d`), matches up the four texture coordinates to the four corners specified by the `vertices[]` array (each vertex of the textured object needs to have a texture coordinate assigned to it), and blends it using the values specified above.

If you run the project, you will notice how the texture is also picking up the colors from the vertices. Comment out the line `gl.glColorPointer(4, GL10.GL_UNSIGNED_BYTE, 0, mColorBuffer)`, and you should now see the texture in its original form.

Now we can replicate some of the examples in the first part of this laboratory. The first is to pick out only a portion of the texture to display. Change `textureCoords` to the following:

```
float[] textureCoords =
{
    0.0f , 0.0f,
    0.5f , 0.0f,
    0.0f , 0.5f,
    0.5f , 0.5f
};
```

The mapping of the texture coordinates to the real geometric coordinates looks like Figure 12. Spend a few minutes to understand what is happening here if you're not quite clear yet. Simply put, there's a one-to-one mapping of the texture coordinates in their array with that of the geometric coordinates.

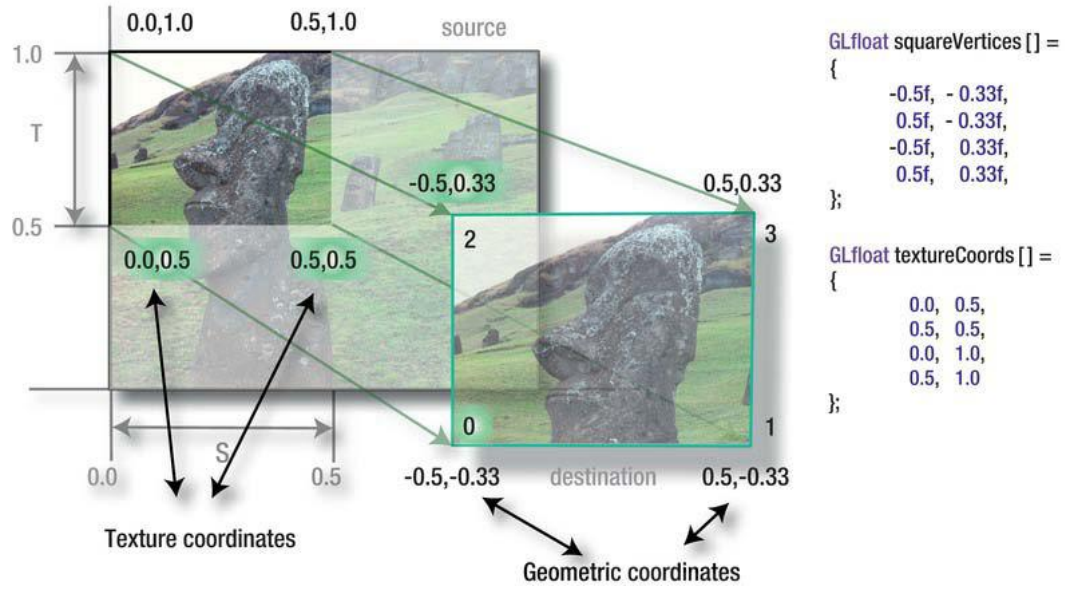

Figure 12: The texture coordinates have a one-to-one mapping with the geometric on

Now change the texture coordinates to the following.

```
float[] textureCoords =
{
    0.0f , 2.0f,
    2.0f , 2.0f,
    0.0f , 0.0f,
    2.0f , 0.0f
};
```

Now let's distort the texture by changing the vertex geometry, and to make things visually clearer, restore the original texture coordinates to turn off the repeating:

```
float vertices[] =
{
    -1.0f, -0.7f,
    1.0f, -0.30f,
    -1.0f, 0.70f,
    1.0f, 0.30f,
};
```

This should pinch the right side of the square and take the texture with it.

Armed with all of this knowledge, what would happen if you changed the texture coordinates dynamically? Add the following code to `draw()` — anywhere should work out.

```
textureCoords[0]+=texIncrease;
textureCoords[1]+=texIncrease;
textureCoords[2]+=texIncrease;
textureCoords[3]+=texIncrease;
textureCoords[4]+=texIncrease;
textureCoords[5]+=texIncrease;
textureCoords[6]+=texIncrease;
textureCoords[7]+=texIncrease;
```

And make both `textureCoords` and `texIncrease` instance variables.

This will increase the texture coordinates just a little from frame to frame. Run and be amazed. This is a really simple trick to get animated textures. A marquee in a 3D world might use this. You could create a texture that was like a strip of movie film with a cartoon character doing something and change the s and t values to jump from frame to frame like a little flipbook. Another is to create a texture-based font. Since OpenGL has no native font support, it's up to us to add it in ourselves. This could be done by placing of the characters of the desired font onto a single mosaic texture and then selecting them by careful use of texture coordinates.